

Quick Start Guide — Decoder

Get a DCC decoder running in minutes

TI MSPM0G3507 LaunchPad + Code Composer Studio / Theia edition. OpenDccCLib itself runs on any microcontroller with a C compiler and an edge interrupt; this guide covers one concrete path.

Generated 2026-09-25 from the repository sources by documentation/guides/build_pdfs.py

Table of Contents

1. What is a DCC Decoder?	3
1.1 Key Terms	3
2. What You Need	4
2.1 Hardware	4
2.2 Signal Pins	4
2.3 Software	4
3. Project Setup	5
4. Understanding dcc_user_config.h	6
5. Building and Flashing	7
5.1 What Happens at Power-On	7
6. How the Decoder Works	8
7. Customizing Callbacks	9
7.1 Terminal Commands	9
8. What's Next	10

1. What is a DCC Decoder?

A DCC decoder sits in a locomotive or an accessory, listens to the DCC signal on the rails, recognizes the commands addressed to it, and drives motors, lights, servos, solenoids or sound.

OpenDccCLib does the protocol work: it classifies bits from edge timing, assembles and validates packets, matches locomotive addresses against the CVs, and dispatches commands to your callbacks. You write the code that moves the hardware.

1.1 Key Terms

- **Bit decoder:** tells one-bits (58 μ s halves) from zero-bits (longer halves; the threshold is 80 μ s) by edge timing.
- **Packet decoder:** assembles bytes, checks the XOR byte, matches the address, dispatches.
- **Configuration Variable (CV):** a numbered setting stored in the decoder: address, speed curve, function mapping.
- **Service mode:** programming-track mode; the decoder answers a successful command with a 6 ms current pulse (ACK).
- **Fail-safe:** with no multifunction packet for this decoder (or broadcast) for CV 11 $\times$ 100 ms, the library calls `on_failsafe_entered`; your callback stops the outputs.

2. What You Need

2.1 Hardware

- TI MSPM0G3507 LaunchPad (LP-MSPM0G3507)
- A DCC source: a commercial command station through a level converter, or a second LaunchPad running the command-station example
- For a real track: an input circuit that turns track-level DCC into 3.3 V logic (optocoupler or divider)
- A USB cable; optionally a motor driver, LEDs or servos to see the outputs

2.2 Signal Pins

The pin assignments come from the project's SysConfig file.

LaunchPad pin	Function
PB1	Main-track DCC in, edge interrupt; from command station PB1
PB4	Service-track DCC in, edge interrupt; from command station PB4
PB12	ACK out; drives the current load the command station senses; to CS PB12
PB17	Track-select in; from CS PB17. Low = listen on PB1, high = listen on PB4. No pull resistor, so connect it or tie it low
PB3	Toggles on each accepted edge (logic-analyzer aid)
PB22	Heartbeat LED
PA10 / PA11	UART TX / RX to the on-board debug probe, 230400 8N1

Two-board setup: CS PB1 to decoder PB1, CS PB4 to decoder PB4, decoder PB12 to CS PB12, CS PB17 to decoder PB17, and GND to GND.

2.3 Software

- TI Code Composer Studio or CCS Theia, with SysConfig
- A serial terminal at **230400 baud**, 8N1
- The OpenDccCLib source tree

3. Project Setup

The example is at `applications/ti_theia/mspm03507_launchpad/decoder/`. Import it in CCS (File > Import > CCS Projects); it reaches the library through the `dcc_lib` symlink to `src/dcc/`.

```
decoder/
  decoder.c                <- main entry: config struct, edge ISR, ring buffer, main loop
  dcc_user_config.h        <- role flags and limits
  decoder_command_parser.c/h <- UART command-line interface
  application_callbacks/
    callbacks_dcc.c/h      <- CV storage stub, CV29 feature mask, command callbacks
  application_drivers/
    ti_driverlib_dcc_driver.c/h <- timestamp, interrupt mask, RailCom delay
    ti_driverlib_uart_driver.c/h <- UART I/O
    ack_pulse_driver.c/h    <- ACK current load
  dcc_lib/                 <- the library (symlink; do not edit)
  Debug/ti_msp_dl_config.c/h <- SysConfig-generated init
```

4. Understanding dcc_user_config.h

```
// #define DCC_COMPILE_COMMAND_STATION      /* not used in this demo */
#define DCC_COMPILE_DECODER
#define DCC_COMPILE_RAILCOM                // RailCom Tx (comment out to strip); inert until wired
#define USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH 8 // deferred dispatch queue; holds depth - 1 packets,
```

Do not define DCC_COMPILE_COMMAND_STATION in a decoder project unless the device really is both, as a booster or repeater would be.

5. Building and Flashing

1. Build (Project > Build, or Ctrl+B).
2. Connect the LaunchPad over USB and flash (Run > Debug, or F11).
3. Open a terminal at 230400 baud. The banner reads DCC Decoder - MSPM0G3507 LaunchPad.

5.1 What Happens at Power-On

`main()` initializes the drivers and the CV defaults (short address 3: CV 1 = 3, CV 29 = 0x06), hands the config struct to `DccConfig_initialize()`, which reads the addressing CVs, then loops. The decoder listens on PB1 while PB17 reads low and on PB4 while it reads high; PB17 has no pull resistor, so connect it or tie it low.

```
while (1) {
    _drain_edge_buffer();           // ISR timestamps -> bit decoder
    DccConfig_run();               // packet dispatch, callbacks, ACK timing, fail-safe
    CallbacksDcc_drain();         // RECV log lines -> UART
    TI_UartDriver_echo_process();
    DecoderCommandParser_process();
}
```

6. How the Decoder Works

- **Edge capture (interrupt).** Every edge on the selected input stores a microsecond timestamp in a 256-entry ring (a full ring drops the edge). Beyond reading the track-select pin and toggling PB3 the interrupt does nothing else, so it stays well under the 58 μ s half-bit.
- **Bit decoding (main loop).** The ring is drained into `DccConfig_decoder_edge_isr()`. An interval under 80 μ s is a one-bit half, 80 μ s or more a zero-bit half.
- **Packet decoding.** Ten or more one-bits arm the preamble; bytes follow until the end bit, and the raw packet is queued. The XOR check and the address match (CV 1 or CV 17–18 as CV 29 bit 5 selects; the accessory CVs 513/521/541 when CV 541 bit 7 is set) happen in the main loop.
- **Dispatch.** `DccConfig_run()` takes queued packets and calls your command callbacks, so they never run inside an interrupt. The one exception is `on_railcom_request`, which runs on the edge path so the reply is ready for the cutout.

Every decoded command is echoed on the terminal as a RECV line, for example `RECV SPEED addr=3 speed=64 dir=FWD mode=128`.

7. Customizing Callbacks

All application logic lives in `callbacks_dcc.c`. The demo logs to the UART; replace the bodies with your motor, light and servo control.

```
void CallbacksDcc_on_speed_command(uint16_t address, uint8_t speed,
                                   bool direction, dcc_speed_mode_enum mode) {
    /* speed: 0 = stop, 2.. = steps (e-stop arrives in on_emergency_stop_command); direction: true = forward,
    set_motor_speed(speed, direction);
}

void CallbacksDcc_on_function_command(uint16_t address, uint8_t function_number, bool state) {
    if (function_number == 0) set_headlight(state); /* F0 = headlight */
}
```

Callback	Fires on
<code>on_speed_command</code>	Speed and direction for this address
<code>on_emergency_stop_command</code>	Emergency stop, addressed or broadcast
<code>on_function_command</code>	Function on or off, F0–F68
<code>on_accessory_basic_command</code> , <code>on_accessory_extended_command</code>	Turnout, signal aspect. Delivered for every accessory packet on the track; compare the address with yours in the callback, the library does not filter accessory addresses
<code>on_cv_write_command</code> , <code>on_cv_verify_command</code> , <code>on_cv_bit_command</code>	Loco CV access, main track (POM) and programming track; the <code>service_mode</code> flag tells them apart. Accessory operations-mode CV access is not delivered in this release
<code>on_consist_command</code>	Consist set or clear, as a notification only; the library does not store CV 19 or match consist addresses
<code>on_binary_state_short_command</code> , <code>on_binary_state_long_command</code> , <code>on_analog_function_command</code>	Binary state (feature expansion) and analog function (advanced operations)
<code>on_failsafe_entered</code> , <code>on_failsafe_exited</code>	No multifunction packet for this decoder or broadcast within CV 11 × 100 ms; such a packet resumed. Your callbacks stop and restart the outputs
<code>cv29_apply_supported_features</code>	Required: clear the CV 29 feature bits this product does not implement

7.1 Terminal Commands

`ADDR <n> <SHORT|LONG|ACC|ACCE>` sets the address, `CLEAR` flushes the log, `ACK ON|OFF` gates the library's ACK pulses, `ACK <width_us>` and `ACK TEST [count]` fire test pulses of your own width (the library's own pulse is fixed at 6 ms, and an ACK requested while one is active is ignored), `STATUS` shows the address last set with `ADDR` (not the CVs), `HELP` lists everything.

8. What's Next

- **RailCom replies.** Add a current-source circuit on a GPIO, set `railcom_tx_pin_set` and a `railcom_delay_us` accurate to 1 μ s (the library asks for 4, 33 and 80 μ s), add in SysConfig the 20 MHz delay timer (`DELAY_TIMER_INST`) the shipped driver expects, and call `DccConfig_decoder_edge_isr()` from the edge interrupt itself instead of draining a buffer, because the reply is timed from the end-bit edge. The library then sends the Channel 1 address broadcast on its own after commands addressed to this decoder and asks `on_railcom_request` for Channel 2 data. CV 28 and CV 29 bit 3 are not consulted. Known limitation: a reply carrying only Channel 2 data is misread as Channel 1 by this library's own command station.
- **Persistent CVs.** Replace the RAM stub behind `cv_read` / `cv_write` with Flash or EEPROM so settings survive a power cycle.
- **Another MCU.** Copy the project and rewrite the driver files. Without RailCom the only interrupt requirement is a microsecond timestamp on each DCC edge; with RailCom the edge interrupt must call the library directly.
- **Developer Guide.** The companion Developer Guide, Decoder, covers the config struct, bit and packet decoders, CV storage, service mode, RailCom replies and fail-safe in depth.